Gradient-Based Optimization and Regularization

Assistant Lecturer: Eng. Ahmed Métwalli

Notation recap.

- θ_t : value of the parameter after iteration t (with $t = 0, 1, 2$).
- g_t : gradient of the loss with respect to θ at iteration t.
- η : learning rate (step size).
- β : decay / momentum coefficient used in moving averages.
- v_t : *velocity* in momentum SGD (moving average of gradients).
- s_t : running average of squared gradients in RMSProp.
- m_t, v_t in Adam : first and second moment estimates of the gradient (note that this v_t is a *different* quantity from the momentum velocity; many libraries overload the same letter).
- $\hat{m}_t, \hat{v}_t$: bias-corrected versions of m_t and v_t.
- λ : L_2 regularization (weight decay) strength.
- h : pre-dropout activation of a hidden unit.
- p : dropout probability (probability of *dropping* the unit).

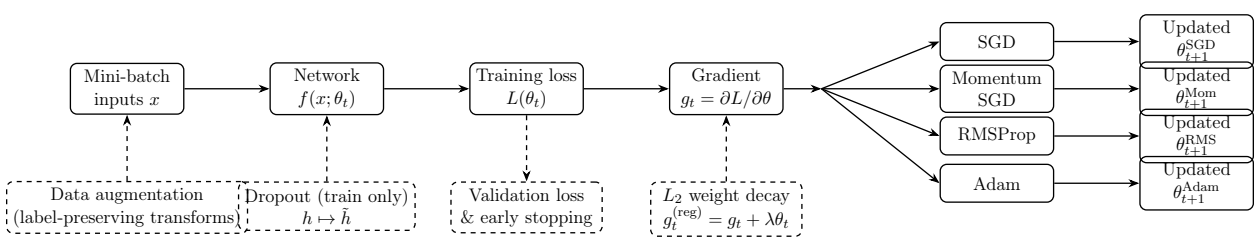

Figure 1: Block diagram of the training pipelines. The top row shows the shared forward/backward path from mini-batch inputs to the gradient g_t. The gradient then fans out to four optimizers (SGD, Momentum SGD, RMSProp, Adam), each producing its own updated parameters $\theta_{t+1}^{(\cdot)}$. Dashed boxes indicate regularization mechanisms applied at different stages of the pipeline.

Where this fits in the course

In a deep neural network all trainable weights and biases are collected into a parameter vector θ. For a mini-batch of training data the network output and the loss can be written as

$$L(\theta) = \text{loss}(\text{network}(\text{inputs}; \theta),\ \text{targets}).$$

Backpropagation gives the gradient

$$g_t = \frac{\partial L(\theta_t)}{\partial \theta},$$

which indicates in which direction θ should be changed to reduce the loss.

Optimizers are update rules of the form

$$\theta_{t+1} = \theta_t + (\text{some function of } g_1, \ldots, g_t).$$

In this sheet that update is studied in *one dimension* (a single scalar parameter) so that the behaviour of different optimizers can be seen clearly.

In a real network:

- θ is a long vector of all weights; the same formulas are applied component-wise.
- Momentum, RMSProp, and Adam keep extra state per parameter (e.g. v_t, s_t, m_t) to smooth or rescale gradients.
- L_2 weight decay adds a penalty $\frac{\lambda}{2}\|\theta\|_2^2$ so that large weights are discouraged.
- Dropout and data augmentation are applied only during training, while early stopping looks at the validation loss and decides when to stop the whole loop.

A minimal training loop looks like:

```
for each epoch:
    for each mini-batch:
        forward pass:   x -> network(x; theta) -> loss L
        backward pass:  compute gradient g = dL/dTheta
        optimizer step: update Theta using SGD / Momentum / RMSProp / Adam
```

The numerical example below fixes g_1 and g_2 by hand so that the focus remains on the *optimizer step* and the effect of regularization.

Worked Example: Optimizers and Regularization (All-in-One)

Goal of this exercise. The same tiny training scenario (one parameter θ and two gradient values g_1, g_2) is used to:

- compare the update behaviour of SGD, Momentum, RMSProp, and Adam;
- see how L_2 weight decay modifies an SGD step;
- recall how dropout, data augmentation, and early stopping act as regularizers.

Everything happens in 1D so that the effect of each choice can be seen clearly.

Problem

Consider a very simple model with a single scalar parameter $\theta \in \mathbb{R}$. During training, the (unregularized) gradient of the loss with respect to θ on a fixed mini-batch is observed over two successive iterations as

$$g_1 = 4, \qquad g_2 = 2.$$

Assume the initial parameter and all optimizer states are zero:

$$\theta_0 = 0, \quad v_0 = 0, \quad s_0 = 0, \quad m_0 = 0.$$

The goal is to compare how different optimizers and regularization strategies modify the parameter updates.

Unless otherwise stated, use learning rate $\eta = 0.1$, momentum / decay coefficients $\beta = 0.5$, and numerical approximations

$$\sqrt{6} \approx 2.45, \qquad \sqrt{8} \approx 2.83.$$

These are *toy* hyperparameters chosen only to keep the arithmetic simple on paper; they are not the usual values used in large neural networks.
In practice, Adam is often used with $(\beta_1, \beta_2, \epsilon) = (0.9, 0.999, 10^{-8})$ and RMSProp with a small $\epsilon \approx 10^{-8}$ for numerical stability. In this toy example much smaller β values and $\epsilon = 0$ are used simply to make the hand calculations easy.

(a) **Vanilla SGD.** Using plain stochastic gradient descent (SGD),

$$\theta_{t+1} = \theta_t - \eta\, g_t,$$

compute θ_1 and θ_2.

(b) **SGD with Momentum.** Momentum SGD maintains a velocity

$$v_t = \beta v_{t-1} + (1-\beta) g_t, \qquad \theta_{t+1} = \theta_t - \eta\, v_t,$$

with $\beta = 0.5$. Compute $v_1, \theta_1, v_2, \theta_2$.

(c) **RMSProp.** RMSProp keeps a running average of squared gradients

$$s_t = \beta s_{t-1} + (1-\beta) g_t^2, \qquad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t}}\, g_t,$$

with $\beta = 0.5$ and ϵ neglected for simplicity (in practice a small $\epsilon \approx 10^{-8}$ is added inside the square root). Compute $s_1, \theta_1, s_2, \theta_2$.

(d) **Adam (with simplified hyperparameters).** Adam combines momentum and RMSProp with bias corrections. Use

$$\beta_1 = \beta_2 = 0.5, \quad \eta = 0.1,$$

and the update

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \\ \hat{m}_t &= \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \\ \theta_{t+1} &= \theta_t - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t}}. \end{aligned}$$

Compute $m_1, v_1, \hat{m}_1, \hat{v}_1, \theta_1$ and $m_2, v_2, \hat{m}_2, \hat{v}_2, \theta_2$ (approximately). In real deep networks Adam is usually run with $\beta_1 = 0.9,\ \beta_2 = 0.999,\ \epsilon \approx 10^{-8}$; the smaller β values here are chosen only so that the numbers stay easy to compute by hand.

(e) L_2 **Weight Decay.** Now an L_2 regularization term $\frac{\lambda}{2}\theta^2$ is added with $\lambda = 0.1$. The regularized gradient becomes

$$g_t^{(\text{reg})} = g_t + \lambda\theta_t.$$

Using *plain* SGD with learning rate $\eta = 0.1$ and the same $g_1 = 4, g_2 = 2$ as above, compute the regularized second parameter value $\theta_2^{(\text{reg})}$ starting from $\theta_0 = 0$. Compare $|\theta_2^{(\text{reg})}|$ to $|\theta_2|$ from part (a).

(f) **Dropout.** A single hidden unit has pre-dropout activation $h = 5$. A dropout layer with drop probability $p = 0.4$ is applied during training.

(i) State the probability that the unit is active during training.

(ii) State the expected activation at test time if a simple "inference scaling" rule is used (multiply by $1 - p$ at test time).

(Remark: many modern libraries use *inverted dropout*, where scaling by $1/(1 - p)$ happens during training instead, and no scaling is needed at test time. Both conventions are equivalent in expectation.)

(g) **Data Augmentation and Early Stopping (conceptual).** Answer briefly:

(i) In one sentence, describe how data augmentation changes the effective training distribution.

(ii) In one sentence, describe how early stopping acts as a form of regularization.

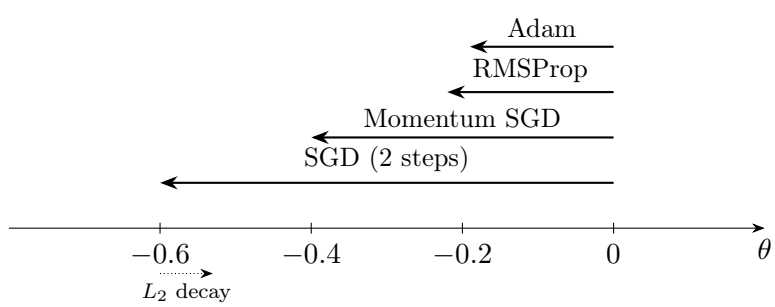

Figure 2: One-dimensional picture of this exercise. All arrows start from the same initial parameter $\theta_0 = 0$ and end at the value of θ_2 computed in parts (a)–(d). SGD takes the longest step to $\theta_2 = -0.60$; Momentum SGD reaches $\theta_2 = -0.40$; RMSProp and Adam take shorter, adaptively scaled steps to about -0.22 and -0.19. The dotted arrow under the axis illustrates L_2 weight decay, which nudges θ_2 slightly back towards 0.

Figure 2. The horizontal line is the parameter axis for θ. Each horizontal arrow shows how far *one optimizer* moves the parameter in two steps, all starting from the same initial point 0. The more aggressive the optimizer, the longer the arrow to the left. The dotted arrow at -0.6 indicates how L_2 decay acts like a small extra push back towards the origin.

Solution

(a) Vanilla SGD. The update rule is

$$\theta_{t+1} = \theta_t - \eta g_t, \quad \eta = 0.1.$$

At $t = 0$:

$$\theta_1 = \theta_0 - \eta g_1 = 0 - 0.1 \cdot 4 = -0.4.$$

At $t = 1$:

$$\theta_2 = \theta_1 - \eta g_2 = -0.4 - 0.1 \cdot 2 = -0.4 - 0.2 = -0.6.$$

Therefore:

$$\boxed{\theta_1 = -0.4, \quad \theta_2 = -0.6.}$$

(b) SGD with Momentum. The momentum rule is

$$v_t = \beta v_{t-1} + (1-\beta) g_t, \qquad \theta_{t+1} = \theta_t - \eta v_t,$$

with $\beta = 0.5$, $v_0 = 0$.

At $t = 1$:

$$v_1 = 0.5 \cdot v_0 + 0.5 \cdot g_1 = 0.5 \cdot 0 + 0.5 \cdot 4 = 2, \qquad \theta_1 = \theta_0 - \eta v_1 = 0 - 0.1 \cdot 2 = -0.2.$$

At $t = 2$:

$$v_2 = 0.5 \cdot v_1 + 0.5 \cdot g_2 = 0.5 \cdot 2 + 0.5 \cdot 2 = 1 + 1 = 2, \qquad \theta_2 = \theta_1 - \eta v_2 = -0.2 - 0.1 \cdot 2 = -0.4.$$

Therefore:

$$\boxed{v_1 = 2,\ \theta_1 = -0.2, \quad v_2 = 2,\ \theta_2 = -0.4.}$$

(c) RMSProp. RMSProp maintains

$$s_t = \beta s_{t-1} + (1-\beta) g_t^2, \qquad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t}} g_t,$$

with $\beta = 0.5$, $s_0 = 0$.

At $t = 1$:

$$s_1 = 0.5 \cdot s_0 + 0.5 \cdot g_1^2 = 0.5 \cdot 0 + 0.5 \cdot 4^2 = 0.5 \cdot 16 = 8, \qquad \sqrt{s_1} = \sqrt{8} \approx 2.83.$$

$$\theta_1 = \theta_0 - \frac{\eta}{\sqrt{s_1}} g_1 = 0 - 0.1 \cdot \frac{4}{2.83} \approx -0.1 \cdot 1.41 \approx -0.141.$$

At $t = 2$:

$$s_2 = 0.5 \cdot s_1 + 0.5 \cdot g_2^2 = 0.5 \cdot 8 + 0.5 \cdot 2^2 = 4 + 0.5 \cdot 4 = 4 + 2 = 6, \qquad \sqrt{s_2} = \sqrt{6} \approx 2.45.$$

$$\theta_2 = \theta_1 - \frac{\eta}{\sqrt{s_2}} g_2 \approx -0.141 - 0.1 \cdot \frac{2}{2.45} \approx -0.141 - 0.1 \cdot 0.82 \approx -0.141 - 0.082 \approx -0.223.$$

Therefore (approximately):

$$\boxed{s_1 = 8,\ \theta_1 \approx -0.141, \quad s_2 = 6,\ \theta_2 \approx -0.223.}$$

(d) Adam. Adam uses

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \\ \hat{m}_t &= \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \\ \theta_{t+1} &= \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t}}, \end{aligned}$$

with $\beta_1 = \beta_2 = 0.5$, $m_0 = v_0 = 0$.

Step $t = 1$:

$$m_1 = 0.5 m_0 + 0.5 g_1 = 0 + 0.5 \cdot 4 = 2, \qquad v_1 = 0.5 v_0 + 0.5 g_1^2 = 0 + 0.5 \cdot 16 = 8.$$

Bias-corrected:

$$\hat{m}_1 = \frac{m_1}{1-\beta_1} = \frac{2}{0.5} = 4, \qquad \hat{v}_1 = \frac{v_1}{1-\beta_2} = \frac{8}{0.5} = 16, \qquad \sqrt{\hat{v}_1} = 4.$$

$$\theta_1 = \theta_0 - \eta \frac{\hat{m}_1}{\sqrt{\hat{v}_1}} = 0 - 0.1 \cdot \frac{4}{4} = -0.1.$$

Step $t = 2$:

$$m_2 = 0.5 m_1 + 0.5 g_2 = 0.5 \cdot 2 + 0.5 \cdot 2 = 1 + 1 = 2, \qquad v_2 = 0.5 v_1 + 0.5 g_2^2 = 0.5 \cdot 8 + 0.5 \cdot 4 = 4 + 2 = 6.$$

Bias-corrected:

$$\hat{m}_2 = \frac{m_2}{1-\beta_1^2} = \frac{2}{1-0.25} = \frac{2}{0.75} = \frac{8}{3} \approx 2.67, \qquad \hat{v}_2 = \frac{v_2}{1-\beta_2^2} = \frac{6}{1-0.25} = \frac{6}{0.75} = 8, \qquad \sqrt{\hat{v}_2} = \sqrt{8} \approx 2.83.$$

$$\theta_2 = \theta_1 - \eta \frac{\hat{m}_2}{\sqrt{\hat{v}_2}} \approx -0.1 - 0.1 \cdot \frac{2.67}{2.83} \approx -0.1 - 0.1 \cdot 0.94 \approx -0.1 - 0.094 \approx -0.194.$$

Therefore (approximately), all requested quantities are:

$$\boxed{\begin{array}{lllll} m_1 = 2, & v_1 = 8, & \hat{m}_1 = 4, & \hat{v}_1 = 16, & \theta_1 = -0.10, \\ m_2 = 2, & v_2 = 6, & \hat{m}_2 \approx 2.67, & \hat{v}_2 = 8, & \theta_2 \approx -0.19. \end{array}}$$

(e) L_2 weight decay with SGD. The regularized gradient is

$$g_t^{(\mathrm{reg})} = g_t + \lambda \theta_t, \quad \lambda = 0.1.$$

The update is

$$\theta_{t+1}^{(\mathrm{reg})} = \theta_t^{(\mathrm{reg})} - \eta\, g_t^{(\mathrm{reg})}.$$

At $t = 0$:

$$\theta_0^{(\mathrm{reg})} = 0, \qquad g_1^{(\mathrm{reg})} = g_1 + \lambda \theta_0^{(\mathrm{reg})} = 4 + 0.1 \cdot 0 = 4, \qquad \theta_1^{(\mathrm{reg})} = 0 - 0.1 \cdot 4 = -0.4.$$

At $t = 1$:

$$\begin{aligned} g_2^{(\mathrm{reg})} &= g_2 + \lambda \theta_1^{(\mathrm{reg})} = 2 + 0.1 \cdot (-0.4) = 2 - 0.04 = 1.96, \\ \theta_2^{(\mathrm{reg})} &= \theta_1^{(\mathrm{reg})} - \eta\, g_2^{(\mathrm{reg})} = -0.4 - 0.1 \cdot 1.96 = -0.4 - 0.196 = -0.596. \end{aligned}$$

From part (a) (unregularized SGD), $\theta_2 = -0.6$. Here $\theta_2^{(\mathrm{reg})} = -0.596$, so

$$|\theta_2^{(\mathrm{reg})}| = 0.596 < 0.6 = |\theta_2|.$$

Thus L_2 weight decay slightly shrinks the parameter magnitude toward zero, demonstrating how it discourages large weights.

(f) Dropout. The unit has activation $h = 5$ before dropout and drop probability $p = 0.4$.

(i) The probability that the unit is *active* during training is

$$P(\text{active}) = 1 - p = 1 - 0.4 = 0.6.$$

(ii) In this toy setting, the "inference scaling" convention is used: all units are kept at test time and activations are multiplied by $(1 - p)$. The expected test-time activation is therefore

$$h_{\text{test}} = (1 - p) \cdot h = 0.6 \cdot 5 = 3.$$

(g) Data augmentation and early stopping.

(i) Data augmentation modifies the effective training distribution by injecting label-preserving transformations, so the model is exposed to a wider variety of inputs around each original example.

(ii) Early stopping acts as a form of regularization by limiting the number of optimization steps based on validation performance, preventing the model from continuing to fit noise in the training set.

Summary of optimizer behaviour in this toy example:

$$\begin{array}{lcl} \text{SGD} & : & \theta_2 = -0.60, \\ \text{Momentum SGD} & : & \theta_2 = -0.40, \\ \text{RMSProp} & : & \theta_2 \approx -0.22, \\ \text{Adam} & : & \theta_2 \approx -0.19. \end{array}$$

These values match the arrows in Figure 2 and show how momentum and adaptive scaling change the effective step size, while L_2 decay, dropout, augmentation, and early stopping help the model generalize better rather than just fitting the training batch.